FrameIQ NEXUS: An Explainable, Diversity-Aware Framework for Automatic Video Thumbnail Selection

Ashpin Geo G A
Div. of Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, Tamil Nadu, India
ashpingeo@gmail.com

Dr. A. Kethsy Prabavathy
Div. of Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, Tamil Nadu, India
kethsy_cse@karunya.edu

***Abstract*—A good thumbnail has to do several jobs at once. It should look decent, say something true about the video, stay readable at a small size, and not repeat the other options on offer. Picking such a frame by hand takes time, and it gets tedious for creators who publish often. This paper describes FrameIQ NEXUS, a prototype that runs entirely on a local machine and combines four steps: shot-aware candidate extraction, interpretable visual-quality scoring, a measure of how well a frame represents its scene, and diversity-aware reranking. The core method, which we call Content-aware Quality-based Thumbnail Ranking (CQTR), uses ordinary computer-vision features. A visual–temporal maximal marginal relevance (MMR) stage then spreads the top suggestions across different parts of the video. If a CLIP-style model is installed, an optional image–text similarity term can rerank frames against a topic supplied by the user. A Flask interface handles uploads, ranked previews, editing and a 3D candidate browser. We describe the implementation and lay out a reproducible protocol for comparing it with simple baselines. The project did not come with finished benchmark numbers or a controlled viewer study, so this paper reports system design and testable hypotheses rather than performance gains we cannot back up.**

***Keywords*—video thumbnail selection, key-frame extraction, shot boundary detection, explainable ranking, maximal marginal relevance, computer vision.**

I. INTRODUCTION

A thumbnail is usually the first thing a viewer sees, and often the only thing they use to decide whether a video is worth their time. For the creator it is a harder choice than it looks. A frame can be technically fine and still tell the viewer nothing. A scene that matters can be spoiled by motion blur, poor exposure or clutter in the background. Taking the middle frame, or the sharpest one, does not settle this. The practical problem is to hand the creator a small set of representative, appealing candidates so they do not have to scrub through the whole video.

Many existing tools treat the task as a single scoring problem, and some lean on large trained models that need a GPU or a cloud service. We wanted something different: a system that runs locally, shows its reasoning, and lets the user steer it. FrameIQ NEXUS is built around three ideas. First, candidates come from detected shots, so one long scene cannot flood the pool. Second, every frame is scored with transparent features, and the breakdown is shown to the user, so a high rank can be explained. Third, the final shortlist is chosen with a diversity-aware step, so the creator is not shown five near-copies of the same moment.

We want to be clear about what this paper is. It is a design and implementation report with an evaluation plan. We do not claim a new visual model, a gain in click-through rate, or an advantage over commercial thumbnail generators. The contribution is an integrated, testable prototype together with a protocol for measuring how much each component adds.

The rest of the paper is organised as follows. Section II reviews related work. Sections III and IV describe the architecture and the ranking method. Section V covers the implementation. Section VI gives the experimental protocol, Section VII discusses privacy and limitations, Section VIII offers some discussion, and Section IX concludes.

II. RELATED WORK

A. Video Summarization and Key-Frame Extraction

Video summarization and key-frame extraction both aim to represent a long sequence with a few images. Classical methods rely on temporal sampling, frame differences, clustering or shot structure. They shrink the search space, but they say little about whether the chosen frame is pleasant to look at. No-reference image-quality cues such as sharpness and exposure address a different part of the problem, and used alone they tend to favour crisp frames that have nothing to do with the story. A thumbnail needs both properties, along with some sense of what the video is about.

Learning-based summarizers have pushed the field further. Zhang et al., for example, modelled the frame sequence with recurrent networks trained on human-made summaries [4]. Such models can capture long-range structure, but they need annotated training data and usually a GPU, and their choices are hard to explain to a non-specialist. Our prototype deliberately stays on the other side of that trade-off. It gives up some modelling power in exchange for transparency, low cost and easy inspection.

On the feature side, the variance of the Laplacian is a common focus measure for judging sharpness [5], and bottom-up saliency models give a cheap way to estimate which regions of a frame draw the eye [6]. We use both ideas as ingredients of the score, not as ends in themselves.

B. Diversity and Semantic Selection

Maximal marginal relevance was introduced for document retrieval and summarisation as a way to balance relevance against novelty in a result set [1]. FrameIQ borrows the idea and applies it to video frames, using lightweight visual descriptors and temporal distance in place of text similarity. For optional topic alignment, the system can use an image–text embedding model compatible with CLIP, which learns a shared space for comparing images and text [2]. We treat this as a relevance aid only. It is not a prediction of what an

audience will click.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the processing workflow. A video is uploaded, split into shots, mined for candidates, scored, reranked for diversity, and returned as downloadable thumbnails.

A. Video Ingestion and Shot Segmentation

The Flask server accepts an uploaded video and reads its metadata and frames with OpenCV. The shot detector compares sampled grayscale frames and combines pixel-difference and histogram-based evidence. Its threshold adapts to each video:

$$T = \mu_d + k\,\sigma_d, \tag{1}$$

where μ_d and σ_d are the mean and standard deviation of the sampled difference scores and k is a configurable constant. The value of k has to be recorded with every experiment. The rule is meant to cope with different amounts of motion and different editing styles, but gradual transitions and fast camera movement can still produce errors.

B. Candidate Extraction and Feature Analysis

Once shot boundaries are found, the pipeline samples frames inside each shot and drops candidates that look too similar. For every frame that remains it computes image-quality cues: Laplacian-based sharpness, brightness, contrast and colour statistics. It also computes content indicators, namely edge density, saliency-related signals, visual centrality and, optionally, whether a face is present. Scene representativeness is estimated from how similar a frame is to the other frames in the same shot. All of these are proxies. They are useful for ranking, but they are not direct measures of human preference.

C. Configurable Parameters

Because the method depends on several tunable settings, every one of them is exposed in configuration and written to the history database alongside the result. Table I lists them. Recording the full set is what makes a run repeatable. Two people using the same video and the same settings should get the same shortlist, and a change in the shortlist should always be traceable to a change in a setting.

TABLE I
CONFIGURABLE PARAMETERS RECORDED WITH EACH RUN

Parameter	Role
k	Sensitivity of the adaptive shot-detection threshold
w_k	Weights of the quality, content and representativeness features
λ	Strength of the redundancy penalty in the CQTR score
β	Quality versus novelty balance in the MMR step
γ	Weight of the optional image–text similarity
k_{out}	Number of thumbnails returned

IV. PROPOSED THUMBNAIL SELECTION METHOD

A. Quality and Content-Aware Ranking

The core CQTR score is a configurable weighted sum of normalised visual signals with a penalty for redundancy:

$$Q(i) = \sum_k w_k\, z_k(i) - \lambda R(i), \tag{2}$$

where $z_k(i)$ is a quality, content or representativeness feature of candidate i and $R(i)$ measures redundancy. The initial weights are engineering choices. They were not learned, and we do not claim they are optimal. Because the score is a plain weighted sum, the contribution of each feature can be shown next to the frame, which helps a user see why a particular image came out on top.

B. Score Normalisation and Fusion

Features live on very different scales, so each one is min–max normalised across the candidate pool before fusion:

$$z_k(i) = \frac{f_k(i) - \min f_k}{\max f_k - \min f_k + \varepsilon}, \tag{3}$$

with a small ε to avoid division by zero. The weights are non-negative and sum to one:

$$w_k \geq 0, \qquad \sum_k w_k = 1. \tag{4}$$

Keeping the weights on a common simplex makes different configurations easy to compare in the ablation studies later on.

C. Diversity-Aware Reranking

The NEXUS extension builds a descriptor for each candidate by concatenating a normalised HSV histogram (18 hue by 12 saturation bins, weight 0.7) with a 32×18 grayscale spatial layout (weight 0.3). Pairwise cosine similarity gives a cheap estimate of visual overlap, and the distance between two candidates is

$$d(i,j) = 1 - \cos(\mathbf{v}_i, \mathbf{v}_j). \tag{5}$$

A novelty term combines this visual distance with normalised temporal separation, so two frames from far apart in the video count as more different than two neighbours. The implementation adjusts the balance between quality and novelty with configurable content-category priors, and it builds the shortlist greedily with an MMR-like rule. For a candidate i and an already selected set S,

$$\text{MMR}(i \mid S) = (1-\beta)\, Q(i) + \beta \min_{j \in S} d(i,j), \tag{6}$$

where the distance term is set to zero while S is empty and β is a diversity control the user can adjust. The set grows one frame at a time:

$$S_{t+1} = S_t \cup \{\arg\max_{i \notin S_t} \text{MMR}(i \mid S_t)\}. \tag{7}$$

This is a heuristic. It encourages variety, but it cannot guarantee that every selected image is semantically distinct, since colour and layout are only a rough stand-in for meaning.

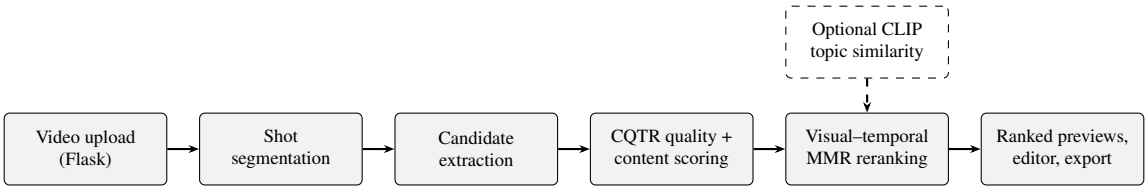


Figure 1. Processing workflow: uploaded video, shot segmentation, candidate analysis, ranking, diversity selection and downloadable thumbnails. The dashed block is only active when the optional model is installed.

D. Semantic Similarity and User Guidance

When the optional dependencies are present, the application encodes the candidate images and a user-supplied topic with a CLIP-based sentence-transformer model. A weighted combination of the visual score and the image–text similarity then reorders a limited subset of candidates:

$$\text{score}(i) = Q(i) + \gamma \text{semantic}(i, \text{prompt}). \tag{8}$$

The standard pipeline works without the model. The web application also offers a thumbnail editor, a results history, side-by-side comparisons and a WebGL candidate "constellation" with an offline fallback. The 3D layout is there to help people explore the options. It does not change how the images are understood.

E. Constraints and Final Selection

The final output is the top-k of the eligible candidates, where eligibility excludes frames removed by the duplicate filter or by minimum quality thresholds:

$$\text{TopK} = \underset{i \in \text{eligible}}{\arg\text{top-k}}\ \text{score}(i). \tag{9}$$

Every returned thumbnail carries its feature breakdown, its shot index and its timestamp, which keeps the result explainable.

Algorithm 1. Greedy diversity-aware selection

Input: candidates C, scores Q, descriptors $\mathbf{v}$, timestamps t, size k, weight β
Output: shortlist S

1. $S \leftarrow \emptyset$
2. **while** $|S| < k$ **and** $C \setminus S \neq \emptyset$:
3. **for each** $i \in C \setminus S$: compute novelty $n(i)$ from $d(i,j)$ and $|t_i - t_j|$ for $j \in S$ ($n(i) = 0$ if $S = \emptyset$)
4. $i^* \leftarrow \arg\max_i (1-\beta) Q(i) + \beta\, n(i)$
5. $S \leftarrow S \cup \{i^*\}$
6. **return** S

The greedy loop costs $O(k|C|)$ similarity lookups once the pairwise matrix is cached, which is small next to the cost of decoding the video, so the reranking step adds little to the total processing time.

F. Illustrative Ranking Example

Table II shows how the reranking step can change an order. The numbers are made up to explain the mechanism and are not measurements. With a diversity weight of 0.3, candidate A (high quality, low novelty) scores $0.7 \times 0.80 + 0.3 \times 0.20 = 0.62$, while candidate B (slightly lower quality, high novelty) scores $0.7 \times 0.70 + 0.3 \times 0.80 = 0.73$. B moves ahead because it adds something the shortlist does not already have.

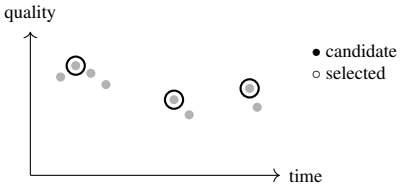


Figure 2. Why diversity helps. A quality-only ranking would pick several of the clustered frames on the left; the MMR step picks one frame per region. The layout is schematic.

TABLE II
ILLUSTRATIVE DIVERSITY RERANKING

Candidate	Quality	Novelty	Combined
A	0.80	0.20	0.62
B	0.70	0.80	0.73

V. IMPLEMENTATION AND ANALYSIS

The repository implements the application in Python with Flask, OpenCV, NumPy and SQLite. Its modules keep video reading, shot detection, key-frame extraction, visual quality, content analysis, representativeness, ranking, evaluation and visualisation separate, so each can be tested or swapped on its own. The NEXUS extension adds the visual–temporal reranker and a Three.js interface. The application supports uploaded-video processing, thumbnail preview and export, optional editing, baseline comparison, ablation inspection and collection of user ratings. By default everything runs locally and needs neither a paid API nor a cloud GPU.

A. Interface and Explainability

The interface is meant to make the ranking inspectable. After processing, the results page lists the shortlisted frames in order, and each one can be expanded to show its component scores, the shot it came from and its timestamp. Fig. 3 sketches the kind of breakdown a user sees. The values in it are invented to show the layout and do not come from a real run.

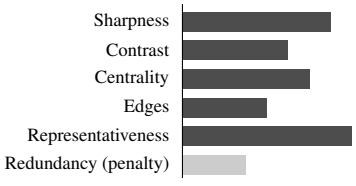


Figure 3. Schematic score breakdown for one candidate (illustrative values only). Dark bars add to the CQTR score; the light bar is the redundancy penalty.

Users can also move the diversity slider, re-run the selection without re-decoding the video, and compare the new shortlist with the earlier one in the history view. The built-in rating form stores a user's preference for each suggestion in SQLite, which gives a straightforward way to collect the human judgments that the protocol in Section VI calls for.

TABLE III
IMPLEMENTATION SCOPE AND LIMITATIONS

Module	Status in FrameIQ NEXUS
Video upload	Implemented in Flask/OpenCV prototype
Shot segmentation	Implemented; thresholds require evaluation
Quality features	Implemented heuristic visual descriptors
Diversity ranking	Implemented configurable reranking
Semantic model	Optional; requires dependencies and validation
Benchmark findings	Not yet established by controlled experiments

Some practical limits are worth stating. The video-processing route is synchronous, so large uploads can delay the response. The optional semantic model may need an initial download and extra memory. The codebase itself notes that comprehensive automated end-to-end runtime testing was not completed, so the presence of finished interface screens should not be read as proof that the system is ready to deploy.

B. A Typical Processing Run

To make the pipeline concrete, consider a ten-minute educational video with a presenter, slides and a few screen recordings. The shot detector finds the cuts between these segments. Frame sampling inside each shot then yields a pool of candidates, and near-duplicates, such as many frames of the same static slide, are removed. The CQTR score favours sharp, well-exposed frames with a clear subject, and the representativeness term prefers frames that resemble the typical content of their shot over unusual transitional frames. Without the diversity step, the top five might all come from the longest talking-head segment. With it, the shortlist is pushed toward the slide material and the screen recordings as well. Whether that behaviour matches what viewers actually prefer is exactly what the evaluation in Section VI is designed to test. We describe it here to explain the intent of the design, not as a result.

VI. EXPERIMENTAL PROTOCOL

A. Research Questions and Video Dataset

The evaluation is organised around four questions. Does shot-aware extraction cover the video better than uniform sampling? Does combining quality and content signals beat quality alone? Does diversity reranking cut redundancy without hurting perceived quality? And does optional semantic reranking improve relevance to a stated topic?

A defensible evaluation needs a documented set of videos covering talking-head, educational, sports and action, travel, and edited multi-scene content. Videos should be partitioned by source so that near-duplicates do not leak between splits. For each video we record its duration, resolution, number of detected shots, number of candidates, processing time and the selected frame. The repository includes comparison functions for five baseline strategies. Their exact configurations should be exported from the running code and named in the final experiment report.

B. Splits and Evaluation Conditions

Table IV lists the conditions to compare. Each row changes one component and holds the rest fixed, so any difference can be traced to that component.

TABLE IV
EXPERIMENTAL CONDITIONS AND METRICS

Condition	Comparison	Outcome
Shot changes	Uniform sampling vs. segmentation	Shot coverage
Ranking	Quality-only vs. combined	Human preference
Diversity	MMR on/off	Redundancy
Semantics	Prompt model on/off	Relevance
Efficiency	Vary video length	Latency

C. Baselines and Ablation Studies

The ablations remove, one at a time, scene representativeness, the content signals, the redundancy penalty and the diversity reranking, while the candidate pool stays fixed. Sweeping the diversity and novelty controls will show whether more variety lowers average perceived quality. The reference points are uniform, middle-frame, random and sharpness-oriented choices, where they apply. Semantic reranking is evaluated separately, because it brings in an additional pretrained model.

D. Metrics and Statistical Analysis

At least three independent raters should judge candidate thumbnails without knowing which method produced them. We suggest 1–5 ratings on visual clarity, relevance to the video, composition, readability at small display size and overall preference. The report should give the number of videos and raters, mean scores with uncertainty intervals, and inter-rater agreement. We would also measure wall-clock processing time and the candidate reduction ratio. Shot-detection precision, recall and F1 need manually annotated boundaries, and thumbnail-selection accuracy needs a justified reference label. Where such annotations are missing, those metrics must be reported as not available. A small blinded user study can show whether people prefer the final suggestions. Any claim about click-through rate would need a separate, properly designed audience experiment.

E. Controlled Comparison Matrix

Table V is the template for the results. Its cells stay empty until the experiments have actually been run.

F. Threats to Validity

Several things could weaken the conclusions of the planned study. Rater preferences are subjective and depend on the rater's background, so the pool of raters should be varied

TABLE V
REQUIRED COMPARISONS (RESULT CELLS TO BE COMPLETED AFTER EXECUTION)

Method	Selection rule	Quality / relevance / runtime
Middle frame	Temporal midpoint	Pending
Random	Uniform random draw	Pending
Sharpness only	Highest Laplacian score	Pending
CQTR	Weighted quality + content	Pending
CQTR + MMR	CQTR with diversity	Pending

and their agreement reported. A small dataset may over-represent one genre, which is why the video set should cover several content types and be stated openly. Hand-chosen weights could be tuned on the same videos used for the final comparison, which would inflate the apparent benefit. To prevent this, tuning should use a held-out development split that is never reused for reporting. Finally, a thumbnail judged in a rating form is not seen the way it is on a real platform, between competing videos and alongside a title. Results from the form should therefore be described as preference judgments, not as predictions of viewer behaviour.

G. Reproducibility Checklist

To keep the study repeatable, the final report should state the exact software versions (Python, OpenCV, NumPy and, if used, the semantic model and its checkpoint), the value of every parameter in Table I, the random seeds used for the random baseline, the hardware on which timings were measured, and the list of videos with their sources and licences. Raw ratings should be released together with the analysis script so that others can recompute the averages and intervals. If a video cannot be shared for licensing reasons, its identifier and the per-video measurements should still be published, so that the result can be checked without the footage.

VII. PRIVACY AND LIMITATIONS

Uploaded videos can contain people, locations and other personal material. In the default configuration, processing happens on the user's own machine, and no video leaves it. The optional semantic model may download weights on first use, but the video frames themselves are not sent anywhere. Even so, a deployed version would need clear rules for how long uploads and generated thumbnails are kept, who can see the results history, and how a user can delete them. Face presence is used only as a content cue and the system does not identify anyone. If the face detector follows the classical cascade approach [3], its known weaknesses with pose, lighting and occlusion apply here too.

The main limitations are these. The scoring weights are hand-chosen. The features are proxies for appeal. Colour-and-layout descriptors cannot capture meaning, so the diversity step can miss two frames that look different but show the same thing. Shot detection can fail on dissolves and rapid motion. The tests so far used a limited set of material, and there are no controlled measurements to support any claim of improvement. These points shape how the results of the planned experiments should be read, and we would rather state them now than discover them later.

VIII. DISCUSSION

Two design choices deserve a comment. The first is the decision to use plain features instead of a trained model. That choice makes the system cheap and transparent, and it lets a creator on an ordinary laptop use it without an account or an internet connection. It also means that the system has no understanding of what is happening in a scene. A frame of a person mid-blink can score as well as a frame of the same person smiling, and the face cue alone will not separate them. The optional semantic stage helps with topic relevance but not with expression or emotion, so we expect the largest gaps between the prototype and a human editor to show up in people-centred content.

The second choice is to expose the diversity control to the user instead of fixing it. A single default cannot suit every video. A lecture with a static camera may need strong diversity to avoid five identical frames, while a short action clip may already have plenty of variety and need little. Leaving the control in the user's hands, and recording it with the result, turns a hard-to-tune constant into a visible part of the workflow. The cost is one more thing for the user to think about, so sensible defaults per content category matter, and the planned sensitivity sweep should tell us how wide a range of values gives acceptable results.

Finally, we should say plainly what the prototype is not. It is not a replacement for editorial judgement and it is not a click-through optimiser. We see it as an assistant that narrows hundreds or thousands of frames to a handful that deserve a human look.

IX. CONCLUSION

FrameIQ NEXUS is a local, explainable prototype for picking video thumbnails. It joins shot-aware candidate extraction, interpretable quality and content scoring, and MMR-style diversity reranking, with an optional topic-aware semantic step and a web interface for review and editing. Its main value, as it stands, is that a user can see why each frame was suggested and can adjust the balance between quality and variety.

The work is still unfinished in one important respect. Nothing here shows that the suggestions are better than simple baselines, and we have deliberately not claimed that. The next step is to run the protocol in Section VI on a documented, source-partitioned video set, report the ablations and the blinded user ratings with their uncertainty, and tune the shot-detection constant k and the weights against what the results show. Making the processing route asynchronous would also remove the most obvious practical limit on larger uploads.

Beyond that, several extensions look worthwhile once the baseline results are in. The hand-set weights could be

replaced by weights fitted to the collected user ratings, which would turn the current heuristics into a lightly supervised model while keeping the breakdown readable. The visual descriptor could be compared with learned embeddings to see whether the extra cost buys better diversity. Text overlays and title suggestions could be added, because a thumbnail rarely works without them. Lastly, a small study with working creators, run on their own videos, would tell us whether the explanations in the interface actually help people choose, which is the question that motivated the prototype in the first place.

REFERENCES

[1] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in *Proc. 21st Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 1998, pp. 335–336.

[2] A. Radford *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, 2021, pp. 8748–8763.

[3] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2001, pp. I-511–I-518.

[4] K. Zhang, W.-L. Chao, F. Sha, and K. Grauman, “Video summarization with long short-term memory,” in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 766–782.

[5] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, “Diatom autofocusing in brightfield microscopy: a comparative study,” in *Proc. 15th Int. Conf. Pattern Recognition*, 2000, pp. 314–317.

[6] L. Itti, C. Koch, and E. Niebur, “A model of saliency-based visual attention for rapid scene analysis,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 11, pp. 1254–1259, Nov. 1998.